UNIVERSIDAD PONTIFICIA BOLIVARIANA

CÓDIGO FUENTE Y ALGORITMOS – SISTEMA FINDYOURPLACE

Juan José Zabala Galvis — T.I. 1042586433

María Angel Vega Medrano — T.I. 1062440296

Salomé Uribe Rengifo — T.I. 1033261155

Matías Tobón García — T.I. 1033188146

2026

Proyecto para optar al título de Técnico Laboral en Programación de Software

Asesor

Cristian Alexander Acevedo Morales

Ingeniera de Software

UNIVERSIDAD PONTIFICIA BOLIVARIANA

TÉCNICO LABORAL EN PROGRAMACIÓN DE SOFTWARE

MEDELLÍN

Contenido

1. Explicación del código fuente y de los algoritmos

1.1 Organización del código

1.2 Script de la base de datos

1.3 Algoritmo de registro y autenticación local

1.4 Algoritmo de autenticación con Google

1.5 Algoritmo de recomendación de destino

1.6 Algoritmo de obtención de imagen real (Wikipedia)

1.7 Algoritmo de detección de intención del chatbot

1.8 Manejo de errores y transacciones

1.9 Pruebas de los algoritmos

Anexo A. Script SQL

Lista de figuras

Figura 1. Diagrama de arquitectura en capas de FindYourPlace.

1. Explicación del código fuente y de los algoritmos

Esta sección explica la estructura del código y los algoritmos principales del sistema. Los algoritmos se presentan primero en pseudocódigo, y luego con el fragmento real de Python que los implementa.

1.1 Organización del código

El código se organiza por capas y, dentro de cada capa, por módulo. Cada capa tiene una única responsabilidad (ver Figura 1):

```
wanderlust_arq/
├── app.py                  Punto de entrada: arma la app y conecta las capas
├── config.py               Configuración (rutas, clave secreta, Google Client ID)
├── models/                 Datos y reglas de negocio
│   ├── usuario.py             Cuentas locales y de Google
│   ├── destino.py             Catálogo + calcular_destino() + obtener_imagen_real()
│   ├── registro.py            Historial en SQLite (guardar_registro, leer_historial_usuario)
│   └── chatbot.py             responder_pregunta(): detección de intención del asistente
├── routes/                 Controladores (Blueprints de Flask)
│   ├── auth.py                /registro, /login, /logout, /auth/google
│   └── main.py                /, /api/recomendar, /api/historial, /api/chat
├── utils/
│   └── decoradores.py         @login_requerido, @api_login_requerido
└── templates/              Vistas HTML
```

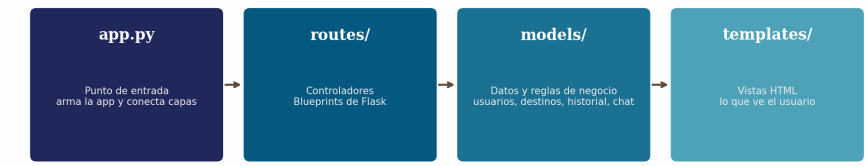


Figura 1. Diagrama de arquitectura en capas de FindYourPlace.

Tabla 1. Capas del código.

1.2 Script de la base de datos

La tabla registros se crea con inicializar_bd() en models/registro.py, la primera vez que se ejecuta la aplicación. Las decisiones más importantes son:

- id como clave primaria autoincremental: cada búsqueda queda identificada sin depender de otro campo.
- Todos los campos de texto (intereses, entorno, destino, pais) se guardan como TEXT: SQLite no obliga tipos estrictos, y esto simplifica guardar listas de gustos como texto separado por comas.
- Un índice sobre usuario (idx_registros_usuario) acelera la consulta del historial, que siempre filtra por ese campo.

1.3 Algoritmo de registro y autenticación local

Las contraseñas se guardan con una función de hash con sal (generate_password_hash de Werkzeug). Al iniciar sesión no se descifra la contraseña: se calcula el hash de lo que la persona escribió y se compara con el guardado.

```
FUNCIÓN iniciarSesion(usuario, clave)
    cuenta ← BUSCAR usuario EN usuarios.json
    SI cuenta = NULO O cuenta.origen ≠ "local" ENTONCES
        RETORNAR error("Usuario o contraseña incorrectos")
    SI NO check_password_hash(cuenta.password_hash, clave) ENTONCES
        RETORNAR error("Usuario o contraseña incorrectos")
    crear sesión con usuario
    RETORNAR redirigir a pagina principal
FIN
```

El fragmento real (models/usuario.py):

```
def verificar_credenciales(usuario, password):
    usuarios = cargar_usuarios()
    cuenta = usuarios.get(usuario)
    if not cuenta or cuenta.get("origen") != "local":
        return False
    return check_password_hash(cuenta["password_hash"], password)
```

El mensaje de error es el mismo si el usuario no existe o si la contraseña es incorrecta, para no revelar qué nombres de usuario están registrados.

1.4 Algoritmo de autenticación con Google

El botón de Google entrega al navegador un token firmado (JWT). El servidor nunca confía en ese token a simple vista: lo envía a verificar contra los servidores oficiales de Google antes de crear la sesión.

```
FUNCIÓN autenticarConGoogle(token)
    SI token vacío ENTONCES RETORNAR error(400)
    SI GOOGLE_CLIENT_ID no configurado ENTONCES RETORNAR error(500)
    TRY
        info ← google_auth.verificar(token, GOOGLE_CLIENT_ID)
    CATCH ValueError
        RETORNAR error("Token de Google inválido", 401)
    correo ← info.email
    SI correo no existe en usuarios.json ENTONCES
        crear cuenta con origen = "google", sin contraseña
    crear sesión con correo
    RETORNAR ok
FIN
```

Complejidad: O(1) además de la llamada de red a Google. La verificación usa la librería oficial google-auth, que valida la firma criptográfica, la audiencia (que el token sea para este Client ID) y que no haya expirado.

1.5 Algoritmo de recomendación de destino

Es el algoritmo central del sistema. Compara los gustos elegidos contra los de cada destino del catálogo, y usa el entorno como filtro adicional cuando el usuario lo especifica.

```
FUNCIÓN calcularDestino(gustosUsuario, entorno)
    candidatos ← []
    PARA CADA d EN DESTINOS HACER
        score ← |gustosUsuario ∩ d.gustos|
        SI score > 0 ENTONCES candidatos.agregar(d con su score)
    SI entorno ≠ "indiferente" ENTONCES
        filtrados ← candidatos DONDE d.tipo = entorno
        SI filtrados no vacío ENTONCES candidatos ← filtrados
    SI candidatos vacío ENTONCES
        RETORNAR aleatorio(DESTINOS)
    maxScore ← máximo score en candidatos
    mejores ← candidatos DONDE score = maxScore
    elegido ← aleatorio(mejores)
    elegido.img ← obtenerImagenReal(elegido.wiki)
    guardarRegistro(usuario, elegido)
    RETORNAR elegido
FIN
```

Elegir al azar entre los "mejores" (en vez de siempre el primero) evita que la recomendación sea siempre la misma para los mismos gustos, sin perder la relevancia del resultado. Complejidad: O(n) sobre el tamaño del catálogo (actualmente más de 30 destinos), suficientemente rápido para no necesitar optimización adicional.

1.6 Algoritmo de obtención de imagen real (Wikipedia)

Para evitar imágenes que no correspondan al destino, primero se busca el título exacto del artículo en Wikipedia, y solo después se pide su foto principal:

```
FUNCIÓN obtenerImagenReal(terminoBusqueda)
    SI terminoBusqueda EN caché ENTONCES RETORNAR caché[terminoBusqueda]
    TRY
        tituloReal ← buscarTituloReal(terminoBusqueda) O terminoBusqueda
        resumen ← GET wikipedia.org/api/rest_v1/page/summary/tituloReal
        SI resumen.status = 200 Y tiene imagen ENTONCES
            caché[terminoBusqueda] ← imagen
            RETORNAR imagen
    CATCH error de red o de datos
        // continúa al respaldo
    caché[terminoBusqueda] ← IMAGEN_RESPALDO
    RETORNAR IMAGEN_RESPALDO
FIN
```

El resultado se guarda en una caché en memoria, así que un destino que ya se buscó antes no vuelve a golpear la red. Si Wikipedia no responde o no encuentra nada, se usa una imagen de respaldo genérica en vez de dejar la pantalla rota.

1.7 Algoritmo de detección de intención del chatbot

El asistente de viajes no usa un modelo de lenguaje: compara la pregunta contra listas de palabras clave por categoría (actividades, transporte, clima, hospedaje, comida, presupuesto), como palabra completa para evitar falsos positivos.

```
FUNCIÓN detectarIntencion(pregunta)
    texto ← normalizar(pregunta)
    PARA CADA (intencion, palabras) EN PATRONES HACER
        PARA CADA palabra EN palabras HACER
            SI palabra aparece como palabra completa en texto ENTONCES
                RETORNAR intencion
    RETORNAR NULO
    // ninguna intención conocida
FIN
```

Cuando no se detecta ninguna intención, el asistente elige al azar una de varias respuestas de "no entendí" (en vez de repetir siempre la misma), y siempre construye la respuesta con el nombre del destino actual, nunca uno anterior.

1.8 Manejo de errores y transacciones

Las rutas /api/* usan un decorador (@api_login_requerido) que responde con un error 401 en formato JSON cuando no hay sesión, en vez de redirigir — una redirección rompería el fetch() del navegador, que espera JSON. Cada inserción en la base de datos (guardar_registro) es una única sentencia SQL, por lo que SQLite la trata como una operación atómica sin necesidad de una transacción explícita.

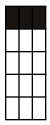


Tabla 2. Manejo de errores.

1.9 Pruebas de los algoritmos



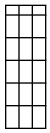


Tabla 3. Casos de prueba de los algoritmos.

Anexo A. Script SQL

```
CREATE TABLE IF NOT EXISTS registros (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    fecha             TEXT NOT NULL,
    usuario           TEXT NOT NULL,
    nombre_viajero    TEXT NOT NULL,
    intereses         TEXT NOT NULL,
    entorno           TEXT NOT NULL,
    destino           TEXT NOT NULL,
    pais              TEXT NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_registros_usuario ON registros (usuario);
```